

McGILL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE

LEARNING WITH SEQUENCE DATA

ASSIGNMENT 4 REPORT

PROF. SIAMAK RAVENBAKSH
COMP 551 - APPLIED MACHINE LEARNING

MATTHEW LACASSE	261183472
ZHENG YE ZHANG	261165863
WILLIAMS LENDJOUNGOU	261167713

14 APRIL 2026

Abstract—This report compares two approaches for text classification on the AG News dataset: a Bidirectional Long Short-Term Memory (BiLSTM) network trained from scratch and a pretrained DistilBERT transformer fine-tuned for the task.

The BiLSTM achieves a peak test accuracy of approximately 93% with 5 million parameters and fast training time, while DistilBERT reaches higher validation performance (up to 94.8%) at the cost of significantly increased computational resources (67 million parameters).

Qualitative error analysis shows that both models struggle with semantically overlapping categories, suggesting an inherent ambiguity in the dataset. Overall, while transformers provide superior performance, LSTMs remain competitive due to their efficiency and simplicity.

I. INTRODUCTION

This assignment has the goal of evaluating sentences and classifying them into 4 different classes: World, Sports, Business, and Sci/Tech. This dataset is the **AG News Classification Dataset**, which provides 120,000 examples of titles and descriptions of headlines, and a test set of 7,600 examples.

To accurately classify the headlines, Recurrent Neural Networks (RNNs) and Transformers are known to perform well on sequences and text analysis. These models expect sequences as inputs—meaning sentences need to be broken down into tokens before being fed into the model. RNNs and Transformers expect a fixed-size sequence, meaning some sentences will need to be cut off, and others padded with a padding character. By pairing these models with a final linear layer, they can predict the class of the headline at high precision.

In this report, we have implemented both models, trained them on the same dataset and tested them. We compare their test score, efficiency and training speed.

II. METHODOLOGY

A. LSTM Pipeline

Recurrent Neural Networks (RNNs) have different architectures: 'Vec2Seq', 'Seq2Vec', 'Seq2Seq' and more. For classifying sentences into classes, Seq2Vec fits our use case. The Seq2Vec architecture takes a sequence as input and transforms it into a fixed-size vector representation. This vector is then passed to a classifier (Linear layer) to produce logits

corresponding to the probabilities of each class.

Inside of a traditional 'Seq2Vec' RNN, the hidden layers are computed as follows:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t) \quad (1)$$

Where $W_{hh}h_{t-1}$ is the contribution from the previous hidden layer, and $W_{xh}x_t$ is the contribution from the input. The output only depends on the final hidden state, however this final hidden state depends on all previous hidden states, making the output inherently depend on the entire forward pass:

$$p(y | x_{1:T}) = \text{Categorical}(y | S(W_{hy}h_T + b_y)) \quad (2)$$

Where h_T is the final state in the RNN, and b_y is the bias.

Certain 'Seq2Vec' RNNs use both a forward and reverse pass to compute the final output. In such case, the hidden state is a concatenation of both the forward and the backward pass. These bidirectional RNNs tend to capture more complexity in the sequences, and are known to be useful for scenarios like text classification tasks.

LSTMs use gating mechanisms where each gate takes the current input and previous hidden state, and together they regulate how the memory is updated and passed forward. An LSTM contains three main gates: an input gate, a forget gate and an output gate.

The memory cells are a combination of an old and a new candidate:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (3)$$

The new hidden state is computed from the updated cell state:

$$h_t = o_t \odot \tanh(c_t) \quad (4)$$

An image of a traditional LSTM model can be found in Fig. 1 [1].

To feed sentences inside of the LSTM, we must break down sentences into token. To do so, we have implemented three different classes.

- 1) **Vocabulary Class:** Takes care of assigning an ID to each new token.
- 2) **Sequence Class:** Vocabulary wrapper keeping the original sentence length without the padding inserts.

- 3) **SequenceFormatter Class:** Allows us to map back and forth from a sentence to a sequence of IDs, and back into a sentence.

The sentences are broken down into tokens using the `get_tokenizer("basic_english")` tokenizer object from `TorchText`, and dropped words with frequencies less than 2. Titles and descriptions are joined together and then split into tokens. We decided to do it this way, prioritizing the headline information and potentially risking the description to be cut off. In the case where a sentence is too long, the sentence is cut down to match the 128 token input size. Length-aware padding was used to specifically meet the input size.

The embedding layer maps discrete, sparse token IDs (integers) into continuous, dense vectors of a fixed size (d_{emb}). As the model trains on the text classification task, it updates the internal weights, placing similar words closer together in the vector space.

The model was analyzed in both unidirectional and bidirectional modes. For the unidirectional model, we only use the last hidden state obtained in the forward pass, and we feed it in a Linear layer to compute the class logits. In the bidirectional mode, we concatenate the final forward and backward hidden states. Since the bidirectional concatenates two hidden states, the linear layer has twice as many inputs.

B. Transformer Pipeline

In task 2, we used the DistilBERT WordPiece tokenizer, instead of using the word-level one from Task 1. WordPiece differs from the tokenizer implemented in Task 1 by operating on sub words rather than whole words. Common words remain intact, but rare or new words are split into known sub words and prefixed with `##`. This eliminates the out-of-vocabulary problem from Task 1 of mapping all unknown words to a `<UNK>` token and losing all information about them. WordPiece also automatically prepends a `[CLS]` token to indicate the start of an input and appends a `[SEP]` token to indicate the end.

The HuggingFace `Trainer` expects objects inheriting from the `Dataset` class, so we developed the `AGNewsDataset` wrapper. This class takes a data set and tokenizes it according to WordPiece. As suggested by the assignment, we truncate each

sequence to a maximum of 128 tokens. Padding is not applied at this stage and is deferred to batch time.

Batch-level padding is done by the `DataCollatorWithPadding` object from the HuggingFace `Transformers` library. This collator dynamically pads each batch to the length of the longest sequence within that batch, rather than padding all sequences to 128 tokens upfront. It also generates a binary attention mask for each sample indicating whether a token is real (1) or padded (0). Altogether, this collator reduces unnecessary computation during training.

We used a 90/10 random split (seed 2026) of the original 120 000 training samples and the test dataset remained untouched. All three datasets are then fed into the `AGNewsDataset` preprocessor yielding 108,000 training samples, 12,000 validation samples and 7,600 testing samples.

We loaded the `distilbert-base-uncased` pretrained model as our transformer backbone using `AutoModelForSequenceClassification` from the HuggingFace `Transformers` library.

According to HuggingFace, DistilBERT is a more compact and faster version of BERT containing around 66 million trainable weights. It was trained on the same corpus used to train Bert (BookCorpus and English Wikipedia) and it approximates BERT's output. [2]

We loaded the DistilBERT model with parameter `num_labels = 4` to attach a linear classification head to the transformer backbone. This maps DistilBERT's output to 4 logit scores (one per AG News class). We also used parameter `dropout = 0.1` for regularization in the classification head. No explicit clipping was done by us.

The model was trained using the HuggingFace `Trainer` on Google Colab using a T4 GPU and the following hyperparameters: 3 epochs, training batch of 16, learning rate of 2×10^{-5} with a linear learning rate schedule warming up over the first 10% training steps, AdamW optimizer with weigh decay of 0.05 and fp16 mixed precision enabled.

The LSTM Tokenization and the Transformer Tokenization are fundamentally different. The LSTM

breaks down sentences into word tokens, whereas the transformer's tokenization breaks down words even more. Here's a small example:

- 1) "Transformers", "are",
"powerful", "models", "!"
- 2) "Transform", "ers", "are",
"power", "ful", "models", "."

III. RESULTS

A. LSTM Results

The unidirectional model was trained for 6 epochs using a batch size of 32, Cross-entropy loss & the Adam optimizer. The following table describes the training loss, validation loss and validation accuracy over each epoch:

TABLE I: Unidirectional LSTM Training

Epoch	Train Loss	Validation Loss	Validation Accuracy
1	1057.3034	68.9239	0.8792
2	553.7945	52.9343	0.9031
3	438.1432	49.6298	0.9121
4	371.0540	48.7135	0.9151
5	317.0305	49.6370	0.9180
6	277.6987	46.1823	0.9236

After training, the model was tested on the test set, and performed at 92.1% accuracy.

1) *Bidirectional Equivalent*: We have also analyzed the performance of the bidirectional equivalent. We used the same loss, optimizer, and parameters to truly compare both models without modifying hyperparameters. The following table describes the training loss, validation loss and validation accuracy over each epoch:

TABLE II: Bidirectional LSTM Training

Epoch	Train Loss	Validation Loss	Validation Accuracy
1	923.4803	78.4506	0.8663
2	490.9764	51.2059	0.9065
3	383.1102	47.8215	0.9156
4	317.1922	49.8227	0.9152
5	266.5853	47.4837	0.9197
6	226.9618	50.7787	0.9212

After training, the model was tested on the test set, and performed at 92.3% accuracy.

2) *Importance of Dropout and Weight Decay*: In both tests, an evident trend of overfitting is observed. For this reason, we also analyze the impact of dropout and weight decay on overfitting. Different models with varying dropout rates in the bidirectional layer were evaluated to maximize performance. The following table shows the final test score for each model at each dropout value (0–0.7):

TABLE III: Test Accuracy for Different Weight Decay Values

Weight Decay	Uni Accuracy	Bi Accuracy
10^{-6}	0.9264	0.9236
10^{-5}	0.9242	0.9247
10^{-4}	0.9238	0.9213
10^{-3}	0.9212	0.9234

TABLE IV: Test Accuracy for Different Dropout Values

Dropout	Bidirectional Accuracy
0.1	0.9161
0.2	0.9197
0.3	0.9278
0.4	0.9263
0.5	0.9299
0.6	0.9282
0.7	0.9186

3) *Pooling Strategy Comparison*: We also compared the default last-hidden-state pooling against mean pooling over all non-padding hidden states (unidirectional LSTM, identical hyperparameters, seed 2026). Last hidden state reached 91.71% test accuracy vs. 91.51% for mean pooling, despite mean pooling attaining a lower final training loss (193.65 vs. 229.51). Averaging appears to dilute the sharper topic-discriminative signal the LSTM concentrates in its final state, suggesting that recency bias is not a real issue on headlines this short.

After analyzing all the models, we concluded that the Bidirectional LSTM Model with 50% dropout performed the best at 93%. Here are some example of misclassified examples:

- 1) British grocer Tesco sees group sales rise 12.0-percent: (Predicted: World, True: Business)
- 2) FBI Interviews Halliburton Whistleblower: (Predicted: Tech/Sci, True: Business)
- 3) Land Rover aims new model at US: (Predicted: Sports, True: Business)
- 4) Democrat Seeks to End Iowa: (Predicted: Sports, True: World)

- 5) Mars water tops science honors: (Predicted: Tech/Sci, True: World)

B. Transformer Results

The DistilBERT transformer with a classification head was trained across 3 epochs with a batch size of 16. The `Trainer` used learning rate of 2×10^{-5} , weight decay of 0.05, the AdamW optimizer and a maximum sequence length of 128 tokens. The model minimized cross-entropy loss over four output classes.

On the one hand, Fig 5 depicts the training loss curve across epochs. The average training loss after the first epoch was 0.2944 and sharply decreased to 0.1506 after the second epoch. Average training loss decreased again after the third epoch to 0.0993, though less aggressively, indicating a decreasing trend.

On the other hand, Fig 6 depicts the validation loss curve across epochs. The average validation loss steadily increased from 0.1883 to 0.1915 from the first to the second epochs. In the third epoch, validation loss rose relatively more sharply to 0.2059. This upward trend suggests signs of overfitting. Although the validation loss is increasing, Fig 7 depicts a steady increase in the validation accuracy from 0.9421 to 0.9481 to 0.9484. The model is improving its overall classification predictions.

TABLE V: Average Training and Validation Metrics across Epochs

Epoch	Train Loss	Val Loss	Val Accuracy
1	0.294357	0.188338	0.942167
2	0.150627	0.191546	0.948167
3	0.099282	0.205856	0.948417

DistilBERT achieved an evaluation accuracy of 94.6% on the held-out test set. Here are some examples of misclassified labels by DistilBERT:

- 1) Card fraud unit nets 36,000 cards: (Predicted: Tech/Sci, True: Business)
- 2) IBM to hire even more new workers: (Predicted: Tech/Sci, True: Business)
- 3) Some People Not Eligible to Get in on Google IPO: (Predicted: Tech/Sci, True: Business)
- 4) Venezuela Prepares for Chavez Recall Vote Supporters and rivals warn of possible fraud: (Predicted: World, True: Business)
- 5) Dell Exits Low-End China Consumer PC Market: (Predicted: Tech/Sci, True: Business)

1) *Impact of Sequence Length:* The `max_length` parameter controls how many subword tokens DistilBERT processes per example. We fine-tuned DistilBERT for one epoch with `max_length` $\in \{64, 128, 256\}$ under otherwise identical conditions.

TABLE VI: Sequence Length vs. Accuracy and Training Time (1 epoch)

<code>max_length</code>	Val. Accuracy	Training Time (s)
64	0.9404	360.5
128	0.9420	421.7
256	0.9426	438.3

Accuracy differences are small (~ 0.2 pp across the full range, see Fig. 4): most AG News headlines already fit within 64 subword tokens, so truncation rarely removes critical information. $64 \rightarrow 128$ gains +0.16 pp at a 17% time cost, while $128 \rightarrow 256$ adds only +0.06 pp — clear diminishing returns consistent with the $O(n^2)$ cost of self-attention. This justifies `max_length` = 128 as the default in our experiments.

IV. DISCUSSION

Here is a brief comparison of our two models:

TABLE VII: Comparison Between Bidirectional LSTM and DistilBERT

Metric	BiLSTM	DistilBERT
Validation Accuracy	92.9%	94.8%
Test Accuracy	92.7%	94.6%
Parameters	5×10^6	67×10^6
Training Time ¹	15s	441s

A. LSTM Discussion

After analyzing the results section, the following table shows the best hyper-parameters and architecture for the LSTM method:

TABLE VIII: Best LSTM Configuration and Performance

Hyperparameter / Metric	Value
Architecture	Bidirectional
Batch Size	32
Dropout	0.5
Learning Rate	10^{-3}
Weight Decay	10^{-4}
Optimizer	AdamW
Loss Function	CrossEntropyLoss
Test Accuracy	93%
Parameters	5×10^6
Training Time ²	15s

1) *Qualitative error analysis (LSTM)*: The validation accuracy and training loss are shown in Fig 2 and Fig 3. The LSTM model lacks an attention mechanism and can therefore struggle with longer sentences where class-identifying information is not repeated. The misclassification distribution shows the LSTM performs worst on **Sci/Tech** (212 errors) and **Business** (185 errors), while performing better on **World** (123 errors) and best on **Sports** (32 errors). This indicates that categories with more ambiguous or overlapping vocabulary—particularly Business and Sci/Tech—are harder to distinguish. In contrast, Sports headlines tend to use more distinctive and consistent terminology, making them easier to classify.

B. Transformer Discussion

TABLE IX: Best Transformer Configuration and Performance

Hyperparameter / Metric	Value
Architecture	DistilBERT
Batch Size	16
Dropout	0.1
Learning Rate	0.02×10^{-3}
Weight Decay	0.05
Optimizer	AdamW
Loss Function	CrossEntropyLoss
Test Accuracy	95%
Parameters	67×10^6
Training Time ³	441s

1) *Qualitative error analysis (Transformer)*: DistilBERT misclassified 403/7,600 test examples (5.30% error rate). The dominant confusion axis is **Business** ↔ **Sci/Tech** (54.8% of all errors): headlines about tech companies (e.g., “*TiVo loss widens*”, “*Microsoft and Dell Get Air Force Pact*”) genuinely straddle both categories. A secondary source (38.2% of errors) involves **World** overlapping with Business or Sci/Tech when political events have financial or scientific implications (“*Deal in Congress to Keep Tax Cuts*”, “*Kerry to reverse stem cell policy*”). This suggests the ~5% error floor reflects genuine label ambiguity in AG News rather than model weakness, further gains would likely require a multi-label formulation rather than a stronger model.

V. CONCLUSION

The error analysis shows both models struggled most along the Business and Sci/Tech axis. The

²Training Time per epoch on an RTX-3090 GPU

³Training Time per epoch on a Google Colab T4 GPU

LSTM had 186 Business and 212 Sci/Tech errors, while DistilBERT had 221 combined errors (54.8% of all errors) across this axis. This is likely because the LSTM lacks direct access to all strongly class-associated words in long headlines at output time and maps unseen words to <UNK>, which is problematic for term-heavy categories like Sci/Tech. Therefore, DistilBERT’s WordPiece tokenizer is a better fit for this task.

Although DistilBERT has over ten times more trainable weights (67M vs 5M), its accuracy is only about 2% higher than the LSTM’s while requiring 30× more training time per epoch (441s vs 15s). Overall, this tradeoff does not favor DistilBERT, as the LSTM achieves comparable accuracy with far lower complexity and training cost.

Adding more regularization to DistilBERT is a possible improvement. As shown in Fig III, validation accuracy improves across epochs, but validation loss increasingly diverges from training loss, indicating overfitting due to near-zero probabilities assigned to the correct class. Exploring techniques like label smoothing to address this would be a useful next step.

VI. STATEMENTS

- 1) **Matthew Lacasse**: Contributed to the LSTM code and report.
- 2) **Williams Lendjoungou**: Contributed to the Transformer code and report.
- 3) **Zheng Ye Zhang**: Contributed to the Transformer code and report.

The LLM “Claude” was used in the following fashion:

- 1) Used to debug the implementation of methods from the `Transformer` library.
- 2) Helped to better understand the overall functionality of DistilBERT

REFERENCES

- [1] S. Ravanbakhsh, “Recurrent Neural Networks,” lecture notes for Applied Machine Learning, McGill University, Montreal, QC, Canada, 2026. [Online]. Available: <https://mycourses2.mcgill.ca/d2l/common/dialogs/quickLink/quickLink.d2l?ou=839340&type=coursefile&fileId=recurrent-neural-networks.pdf> (accessed Apr. 12, 2026).
- [2] Hugging Face, “DistilBERT base uncased,” 2019. [Online]. Available: <https://huggingface.co/distilbert/distilbert-base-uncased> (accessed Apr. 12, 2026).

APPENDIX

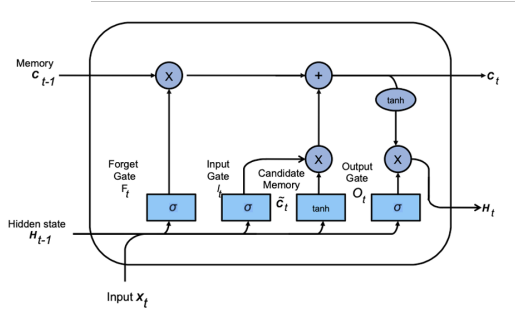


Fig. 1: LSTM Block Architecture [1]

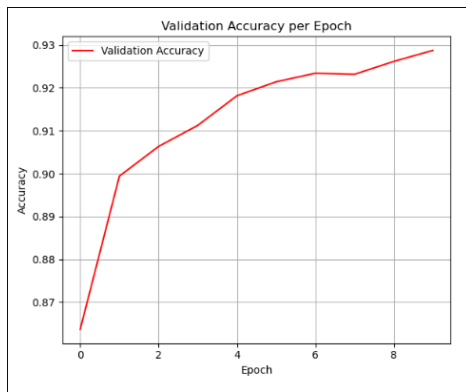


Fig. 2: Bidirectional Accuracy vs Epochs

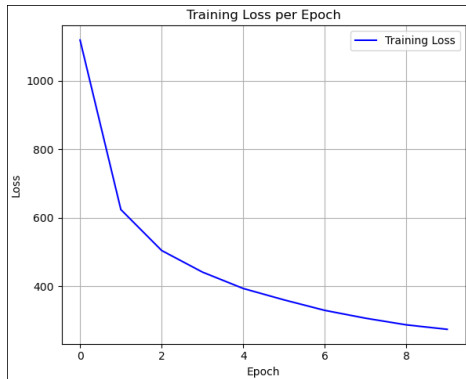


Fig. 3: Bidirectional Loss vs Epochs

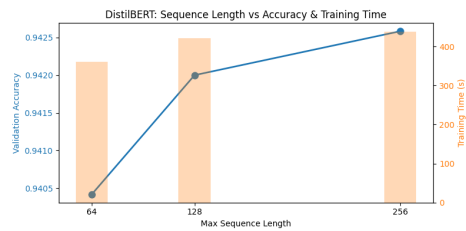


Fig. 4: DistilBERT: sequence length vs. validation accuracy (line) and training time (bars).

Figure 5: Average Training Loss vs Epochs

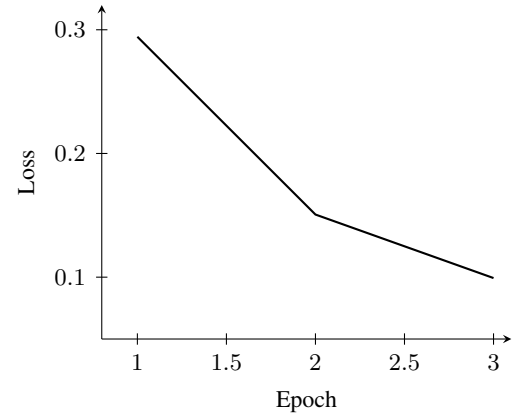


Figure 6: Average Validation Loss vs Epochs

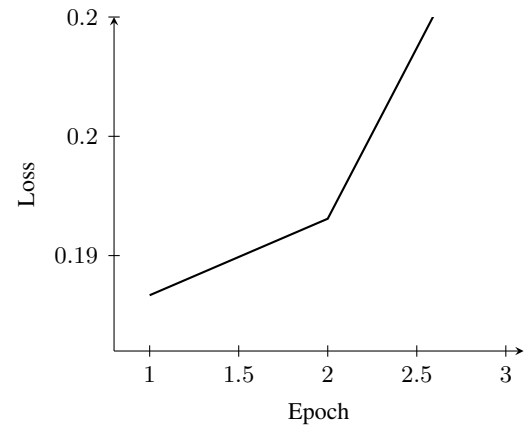


Figure 7: Average Validation Accuracy vs Epochs

